

RESEARCHVAULT

Offline NLP + Reinforcement Learning Research Knowledge Agent

Complete Project Blueprint: architecture, models, parameters, data design, RL environment, evaluation, and implementation roadmap

Prepared as a practical M.Sc.-level implementation specification

Version: September 2026

Contents

1. 1. Project Definition and Goals
2. 2. Why NLP + RL and How They Interact
3. 3. System Architecture
4. 4. Core Components and Libraries
5. 5. Local Model Strategy for Lightweight Hardware
6. 6. PDF Ingestion and Research-Paper NLP Pipeline
7. 7. Chunking, Embeddings, Vector Retrieval, BM25 and Graph Search
8. 8. Reranking, Evidence Selection and Answer Generation
9. 9. Accuracy, Confidence, Threshold Calibration and Abstention
10. 10. Persistent Memory and User Personalization
11. 11. Reinforcement Learning Formulation
12. 12. DQN Environment, State, Actions and Reward Design
13. 13. Recommended Hyperparameters and Starting Configuration
14. 14. Speed and Resource Optimization
15. 15. Project Database and File Structure
16. 16. Evaluation Dataset and Experimental Methodology
17. 17. Baselines, Ablations and Research Questions
18. 18. UI / UX Design and Key Features
19. 19. Research-Gap Detection and Advanced Extensions
20. 20. Step-by-Step Build Roadmap
21. 21. Risk, Failure Modes and Recovery Strategy
22. 22. Deliverables and Dissertation-Ready Outputs
23. 23. References and Official Documentation

1. Project Definition and Goals

ResearchVault is an offline-first research intelligence application. A user uploads one or more research papers; the system extracts and structures the content using NLP, converts the knowledge into persistent semantic and structured representations, and later answers questions using only local knowledge. Reinforcement learning is used to learn which retrieval and evidence-gathering actions should be selected for different query types and user contexts.

Core principle

NLP learns what the papers say. The knowledge base remembers it. Retrieval finds it. The local language model explains it. Reinforcement learning learns how to retrieve and use the knowledge more effectively.

Goal	Definition	Success signal
Offline operation	No internet, cloud search, or external AI API is needed during normal use.	Network disabled and QA still works on the stored corpus.
Persistent knowledge	Uploaded papers remain searchable after the application is restarted.	Knowledge survives process restart and machine reboot.
Research understanding	Paper structure, entities, methods, datasets, metrics, results, and limitations are represented.	Structured extraction is measurable on a labeled subset.
Reliable QA	Answers should be grounded in retrieved evidence and cited to paper/page.	High answer and citation correctness with low unsupported-claim rate.
Adaptive behavior	The system learns better retrieval strategies from feedback.	RL policy reduces error or retrieval cost on held-out tasks.
Lightweight operation	Use small local models and bounded retrieval context.	Acceptable latency and memory use on target hardware.

1.1 Functional Requirements

- Upload PDF research papers individually or in batches.
- Extract text, page numbers, sections, tables, formulas, and figures where possible.
- Build semantic chunks with document metadata preserved.
- Generate local embeddings and maintain a searchable vector index.
- Run lexical BM25 retrieval for exact terms, numbers, model names, and dataset names.
- Build a local knowledge graph of papers, methods, datasets, metrics, results, limitations, and relationships.
- Route queries through an RL policy that can select and sequence retrieval actions.
- Generate answers with a local LLM using only selected evidence.
- Attach page-level or chunk-level citations.
- Estimate answer support/confidence and abstain when evidence is insufficient.
- Collect user feedback and corrections for personalization and RL rewards.

2. Why NLP + RL and How They Interact

The project should separate the responsibilities of NLP, retrieval, generation, memory, and RL. This makes the system easier to measure and prevents reinforcement learning from being used where a deterministic information-retrieval component is more appropriate.

Layer	Primary job	Recommended methods
PDF / document layer	Recover trustworthy text and layout.	PyMuPDF4LLM; selective OCR.
NLP layer	Identify sections, entities, phrases, relations, research claims and metadata.	spaCy, optional SciSpaCy, rule-based research extractors.
Representation layer	Encode semantic meaning.	BGE-small embeddings.
Retrieval layer	Find candidate evidence.	FAISS + BM25 + graph retrieval.
Ranking layer	Order the candidates by query-passage relevance.	Small CrossEncoder reranker.
Generation layer	Produce a concise grounded answer.	Local Qwen-family GGUF model via llama.cpp.
Memory layer	Persist papers, chunks, graph, interactions, feedback and user preferences.	SQLite + FAISS + NetworkX.
RL layer	Choose retrieval strategy/action sequence to maximize answer quality and efficiency.	Gymnasium + Stable-Baselines3 DQN.

2.1 Correct Use of Reinforcement Learning

RL should not be responsible for memorizing the paper or generating every answer token. The most defensible formulation is to make retrieval strategy selection a sequential decision problem. At each step, the RL agent observes the current query and retrieval state, chooses an action such as dense search or graph search, sees the new evidence state, and eventually receives a reward based on correctness, citation support, user feedback, and retrieval cost.

State -> RL policy -> Retrieval action -> New evidence state -> ... -> Answer -> Reward -> Policy update

3. System Architecture

The following architecture keeps all core components local and makes the RL contribution explicit.

ResearchVault - Offline NLP + RL Architecture

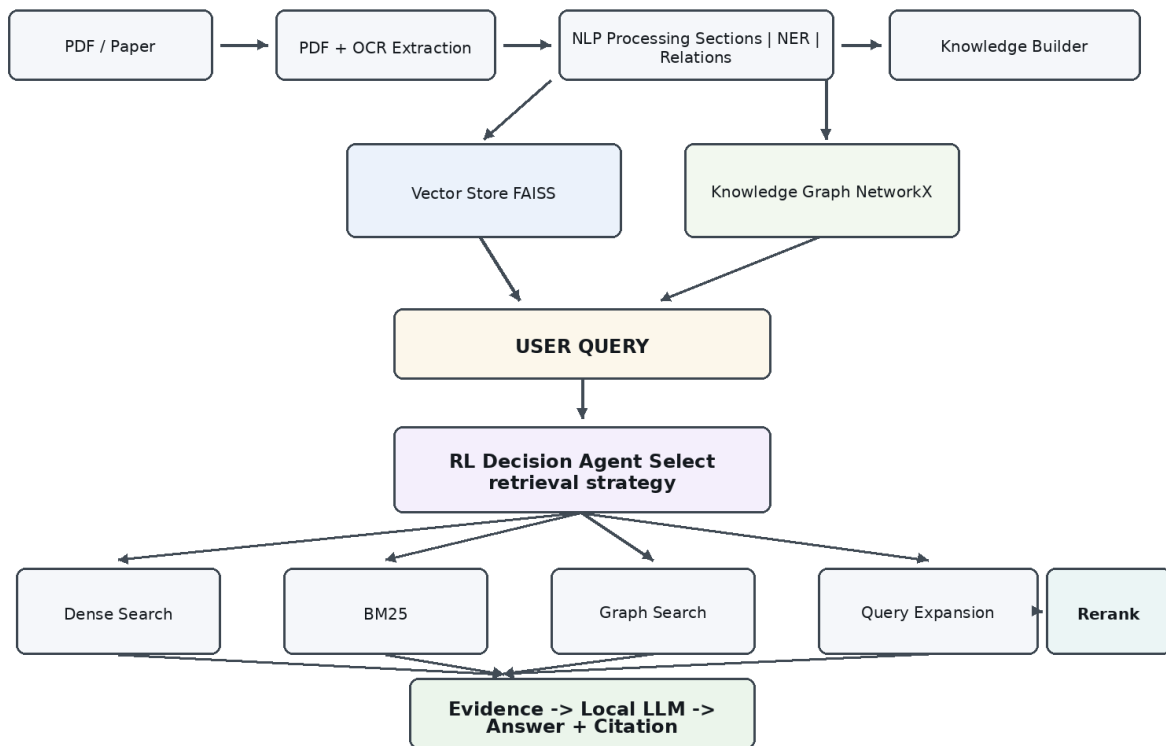


Figure 1. End-to-end ResearchVault architecture. The retrieval actions can be executed locally and in parallel once selected by the policy.

3.1 Query-Time Sequence

1. Normalize the question and detect intent: factual, numerical, methodology, comparison, synthesis, or unanswerable candidate.
2. Encode the query with the local embedding model and extract exact entities/terms.
3. Build the RL state from retrieval scores, graph coverage, question type, previous actions, and user preferences.
4. Let the RL policy select one or more retrieval actions.
5. Run dense search, BM25, graph lookup, or hybrid search as directed.
6. Fuse results using Reciprocal Rank Fusion (RRF) or a learned linear combiner.
7. Rerank only the top candidate set with a CrossEncoder.
8. Apply evidence gates: minimum support, diversity, and claim-level checks.
9. Pass only a small evidence set to the local LLM.
10. Generate a structured answer with citations.
11. Run citation/numeric/grounding verification.
12. Return an answer or abstain, then log feedback for later RL training.

4. Core Components and Libraries

Component	Recommended library/model	Role	Why
PDF parser	PyMuPDF4LLM	Extract structured Markdown/JSON/TXT and page-aware content.	Lightweight, layout-aware, supports OCR selectively.
NLP	spaCy	Sentence splitting, NER, rule patterns, metadata extraction.	Fast local pipeline; easy custom components.
Scientific NLP (optional)	SciSpaCy	Scientific entity extraction and specialized NLP.	Useful for technical vocabulary and research domains.
Embeddings	BAAI/bge-small-en-v1.5	Dense semantic representations.	Small, 384-dimensional embeddings; practical for local use.
Vector retrieval	FAISS	Fast nearest-neighbor retrieval.	Simple, local, high-performance indexing.
Lexical retrieval	rank_bm25 or equivalent BM25 implementation	Exact-term retrieval.	Useful for model names, numbers, dataset names, acronyms.
Graph	NetworkX	Local research knowledge graph.	Simple Python-native graph construction and traversal.
Reranking	cross-encoder/ms-marco-MiniLM-L-6-v2	Query-passage relevance reranking.	Good first accuracy/speed trade-off.
Local LLM	Qwen3.5-2B or 4B GGUF	Grounded answer generation.	Small local model family suited to constrained hardware.
Inference	llama.cpp	Local GGUF inference server.	Quantization, CUDA, CPU+GPU hybrid, local REST API.
RL	Gymnasium + Stable-Baselines3	Environment and DQN training.	Convenient implementation for discrete action spaces.
Database	SQLite	Persistent metadata, feedback, episodes and logs.	Zero-server, fast, portable.
UI	Streamlit	Interactive research UI.	Fast local deployment and easy iteration.
API	FastAPI	Clean service boundary between UI and engine.	Useful when separating components or scaling later.

Current library/model facts are based on the official documentation linked in the References section. PyMuPDF4LLM currently supports Markdown, JSON and TXT extraction, layout analysis, multi-column pages, tables/graphics, page-aware output, and selective OCR. The BGE-small model card reports a 384-dimensional BERT-based embedding model and a 512-token maximum position length. llama.cpp supports quantization, CUDA, CPU+GPU hybrid inference, and a local OpenAI-compatible server. See References for source links.

5. Local Model Strategy for Lightweight Hardware

The target development machine has an NVIDIA RTX 3050 with 4 GB VRAM and 8 GB RAM. For that hardware, start with a small quantized local language model and keep the retrieval pipeline computationally dominant rather than increasing the LLM size prematurely.

Purpose	Start with	Upgrade path	Reason
Answer generation	Qwen3.5-2B, GGUF Q4_K_M	Qwen3.5-4B Q4/Q5 after RAM upgrade	Keeps local generation practical.
Embeddings	BAAI/bge-small-en-v1.5	Larger embedding model only after baseline is stable	Small embedding model is fast and memory-efficient.
Reranking	MiniLM CrossEncoder	BGE reranker if accuracy gain justifies latency	Reranking is only applied to a bounded candidate set.
Inference	llama.cpp	Same stack; tune GPU offload and context	Local server and quantized inference are easy to control.

Hardware-aware rule

On 4 GB VRAM / 8 GB RAM, prioritize a 2B quantized generator, small embeddings, bounded context, caching, and fewer retrieval/LLM calls. A 16 GB RAM upgrade will make 4B-class local experimentation more comfortable. Treat this as an engineering constraint, not a permanent model limit.

5.1 Local LLM Configuration

- Model format: GGUF
- Quantization start: Q4_K_M
- Context: 8192 tokens
- Temperature: 0.10-0.20 for factual QA
- Top-P: 0.80-0.90
- Top-K: 20
- Max new tokens: 700-1200
- Generation passes per question: 1 preferred

The exact model release and quantization should be pinned in a project manifest. Do not hard-code a claim that one model or quantization is universally best: benchmark at least two candidate local generators on the same validation set.

6. PDF Ingestion and Research-Paper NLP Pipeline

Research papers often contain two-column text, tables, figures, formulas, headers/footers, and scanned pages. The ingestion system should preserve page and section context because later citation verification depends on it.

6.1 Extraction Strategy

1. Open the PDF and inspect whether each page has a usable text layer.
2. Use PyMuPDF4LLM layout-aware extraction by default.
3. Use OCR only on pages that need it rather than OCRing the whole paper.
4. Preserve page number, section heading, subsection, and source filename for every chunk.
5. Keep the original PDF immutable; store derived Markdown/JSON separately.
6. Cache extraction output to avoid repeat processing.

6.2 Research-Specific NLP Schema

Entity / claim type	Examples
AUTHOR	researcher names
MODEL / METHOD	U-Net, ResNet-50, SVM, PPO
DATASET	ImageNet, BraTS, KDDTrain+
TASK	classification, segmentation, detection
METRIC	accuracy, Dice, F1, AUROC
RESULT	94.2% Dice, 0.87 AUROC
LOSS_FUNCTION	cross-entropy, Dice loss
OPTIMIZER	Adam, SGD
DOMAIN	medical imaging, cybersecurity
LIMITATION	small dataset, class imbalance
FUTURE_WORK	proposed extension or gap

6.3 Section Classification

- BACKGROUND / MOTIVATION
- PROBLEM / OBJECTIVE
- RELATED WORK
- METHOD / ARCHITECTURE
- DATASET
- EXPERIMENT / SETUP
- RESULT
- DISCUSSION
- LIMITATION
- CONCLUSION
- FUTURE WORK

A first implementation can use section headings and rule-based patterns rather than training a complex classifier. A later version can train a sentence or paragraph classifier on a small labeled research-paper corpus.

7. Chunking, Embeddings, Vector Retrieval, BM25 and Graph Search

7.1 Chunking Starting Values

Parameter	Starting value	Reason / tuning note
Target chunk size	350 tokens	Small enough for precise evidence; keep section context.
Overlap	50 tokens	Preserves continuity across boundaries.
Minimum chunk	100 tokens	Avoids tiny fragments.
Maximum chunk	500 tokens	Prevents oversized context blocks.
Metadata prefix	Paper + section + subsection	Improves semantic retrieval and citation traceability.

These values are starting points. The validation set should compare several configurations, for example 250/40, 350/50, 450/60 tokens, and report retrieval Recall@K and answer support rate.

7.2 Embedding Configuration

Embedding model: BAAI/bge-small-en-v1.5

Dimension: 384

Normalization: True

Batch size: 16 initially; benchmark 8, 16, 32

Store one vector per chunk; persist index to disk

The BGE-small-v1.5 model is 384-dimensional and its model configuration reports 512 maximum position embeddings. Use query/document encoding conventions recommended by the model card and keep embedding preprocessing consistent between indexing and querying.

7.3 FAISS

Corpus size	Index	Starting settings
Up to ~100k chunks	IndexFlatIP with normalized embeddings	Exact inner-product search; simplest baseline.
Larger collections	IndexHNSWFlat	M=24, efConstruction=200, efSearch=64 as starting values.
Very large collections	Consider IVF/HNSW variants after benchmark	Optimize memory and recall only when needed.

FAISS supports HNSW indexes and exposes parameters such as M, efConstruction and efSearch. Higher efSearch can improve recall at the cost of more work, so benchmark it rather than assuming a fixed value is optimal.

7.4 BM25

BM25 starting values:

k1 = 1.2

b = 0.75

Use lexical retrieval especially for exact scientific terms, model names, numbers, acronyms, dataset names, metric names, and quoted phrases.

7.5 Knowledge Graph

PAPER -> PROPOSES -> MODEL

MODEL -> USED_FOR -> TASK

MODEL -> TRAINED_ON -> DATASET

MODEL -> ACHIEVES -> METRIC/RESULT

PAPER -> REPORTS -> RESULT

PAPER -> HAS_LIMITATION -> LIMITATION

PAPER -> CITES -> PAPER

The graph complements vector search; it is not a replacement. A comparison query or a question about exact relationships can benefit strongly from graph traversal.

8. Reranking, Evidence Selection and Answer Generation

8.1 Recommended Retrieval Pipeline

Question

- > Dense retrieval: top 12
- > BM25 retrieval: top 12
- > Graph retrieval: top 10
- > RRF fusion: top 20 candidates
- > CrossEncoder rerank: top 20
- > Evidence selection: top 5-7 chunks
- > Local LLM: answer once

The exact K values should be tuned, but keeping the final evidence set small is important for both speed and model focus.

8.2 RRF

Reciprocal Rank Fusion is a useful first-stage method for combining heterogeneous retrieval lists. It rewards documents that appear near the top across multiple retrieval systems without requiring score calibration across different models.

8.3 Reranker

Option	Use case	Trade-off
cross-encoder/ms-marco-MiniLM-L-6-v2	First implementation	Fast enough to rerank a small candidate set.
BAAI/bge-reranker-base	Accuracy-focused experiment	Heavier; validate whether it materially improves grounded QA.

Do not interpret a CrossEncoder score as a universal probability. The threshold must be calibrated on your own validation data.

8.4 Evidence Rules

- Prefer evidence from the same section as the requested fact when available.
- Prefer direct result paragraphs/tables for numerical questions.
- For comparison questions, require evidence from both sides of the comparison.
- For synthesis questions, require at least two independent source chunks or papers when the claim spans multiple documents.
- Keep page/chunk provenance attached to every evidence item.
- Deduplicate near-identical chunks before generation.

8.5 Grounded Answer Prompt

SYSTEM

You are an offline research knowledge assistant.

Use only the supplied evidence.

Do not invent facts, values, papers, authors, citations or references.

Every factual claim must be supported by evidence.

Numerical claims must match evidence exactly.

If evidence is insufficient, state that the stored collection does not contain sufficient evidence.

EVIDENCE

[Paper | page | section | text]

...

QUESTION

...

9. Accuracy, Confidence, Threshold Calibration and Abstention

There is no universal correct similarity or reranker threshold. The robust approach is calibration: create a labeled development set, measure retrieval and answer outcomes, fit or select thresholds on that set, and freeze them before evaluating the held-out test set.

9.1 Validation Dataset

Question type	Suggested count in a 500-question set
Factual retrieval	100
Numerical / result lookup	75
Methodology	75
Comparison	75
Multi-paper synthesis	75
Unanswerable / insufficient evidence	50
Citation verification	50

The numbers above are a suggested composition, not a requirement. Keep test questions grouped by unseen papers where possible to reduce leakage.

9.2 Confidence Model

Features:

reranker_top1
 reranker_margin
 dense_top1
 dense_top3_mean
 bm25_rank
 entity_overlap
 number_match
 graph_coverage
 evidence_count
 question_type

Calibrator:

logistic regression or small gradient-boosted model

Output:

P(supported_answer)

9.3 Starting Confidence Gates

Calibrated support probability	System behavior
--------------------------------	-----------------

≥ 0.80	Answer normally with citations.
0.65-0.79	Answer with a low-confidence flag and conservative wording.
< 0.65	Abstain or explicitly state insufficient evidence.

Do not treat these as final thresholds

The 0.80 / 0.65 values are initial engineering gates only. Final thresholds must be selected using validation-set precision/recall, unsupported-claim rate, and abstention quality.

9.4 Numeric Claim Gate

For answers containing exact percentages, counts, metrics, dates, or experimental values, require the stated number to be present in the supporting evidence or to be derivable by an explicitly checked arithmetic transformation. Reject or regenerate unsupported numeric claims.

10. Persistent Memory and User Personalization

ResearchVault should store both knowledge and interaction history. This lets the same system adapt to the user without changing the source documents.

Memory type	Examples
Document memory	PDF metadata, paper title, authors, year, page map, extracted sections.
Semantic memory	Embeddings for chunks, summaries, concept representations.
Structured research memory	Methods, datasets, metrics, findings, limitations, relationships.
Interaction memory	Questions, retrieved evidence, answers, citations, feedback.
Preference memory	Preferred answer length, technical depth, favorite topics, frequently used papers.
Correction memory	User corrections and accepted/rejected answers.

10.1 Example User-Learning Behavior

Observed feedback:

User repeatedly requests mathematical detail

User frequently opens page-level citations

User rejects answers without exact evidence

Preference profile becomes:

technical_depth = high

citation_detail = page_level

unsupported_claim_tolerance = low

Personalization should change retrieval/presentation behavior, not alter the source facts. User preferences are a decision signal, not a license to override evidence.

10.2 Learning From Corrections

When a user corrects an answer, save the correction, the prior answer, the evidence used, and the corrected evidence. During future retrieval the system can use this as interaction memory and as labeled feedback for future policy training.

11. Reinforcement Learning Formulation

Markov decision framing

State = current query + retrieval/evidence features + user/context features. Action = choose a retrieval or evidence-management operation. Transition = new evidence state after executing the action. Reward = answer correctness + citation support + user feedback - hallucination penalty - unnecessary retrieval cost.

RL Loop for Adaptive Retrieval

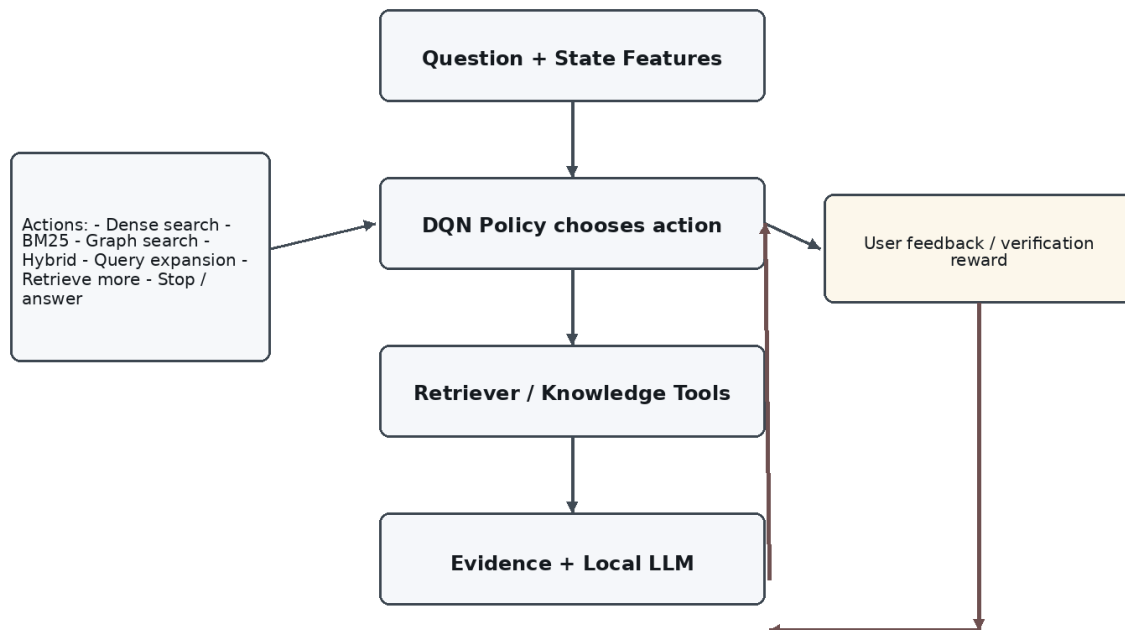


Figure 2. RL loop. The policy learns to choose retrieval actions; the local LLM remains the grounded answer generator.

11.1 Action Space

Action ID	Action
0	Dense semantic retrieval
1	BM25 lexical retrieval
2	Knowledge graph retrieval
3	Hybrid retrieval
4	Query expansion / rewrite
5	Retrieve more evidence
6	Rerank / re-evaluate candidates
7	Stop retrieval and answer
8	Abstain / insufficient evidence

11.2 RL State Features

- Question type / intent encoding.
- Dense top-1 score and top-3 mean score.
- BM25 top rank / lexical overlap.
- Reranker top score and top-score margin.
- Number of relevant candidates so far.
- Entity and relation coverage.
- Graph match indicator.
- Previous action and step count.
- User preference features.
- Whether numeric or citation evidence has been found.

12. DQN Environment, State, Actions and Reward Design

A discrete action space makes DQN a practical first RL algorithm. Stable-Baselines3 currently includes a DQN implementation with an MLP policy for discrete actions. The official documentation and module expose parameters such as learning rate, replay-buffer size, learning starts, batch size, gamma, training frequency, target update interval, and epsilon exploration settings.

12.1 Custom Gymnasium Environment

```

reset()
-> load question
-> initialize retrieval state

step(action)
-> execute retrieval action
-> update evidence features
-> compute intermediate reward
-> done if answer/abstain/max_steps

final reward
-> answer correctness
-> citation correctness
-> support
-> user feedback
-> retrieval cost

```

12.2 Retrieval Horizon

Speed constraint

Start with a maximum of 3 retrieval decisions per question. This keeps the policy bounded and prevents an RL loop from exploding latency. Increase only if the validation data shows a measurable gain.

12.3 Reward Function

A useful initial formulation is:

$$R = 4A + 3C + 2E + U - 3H - L$$

where A = answer correctness, C = citation correctness, E = evidence relevance/support, U = user feedback value, H = unsupported/hallucinated content, and L = latency or unnecessary retrieval cost. Tune these weights with ablations rather than claiming they are universally optimal.

12.4 Concrete Reward Signals

Event	Initial reward
Completely correct answer	+10
Correct citation support	+6
Strong evidence retrieved	+4
User positive feedback	+3
Efficient successful retrieval	+1
Irrelevant retrieval	-5
Unsupported factual claim	-10
Hallucinated claim	-15
Unnecessary retrieval step	-3

13. Recommended Hyperparameters and Starting Configuration

These are recommended starting points for engineering. They are not guaranteed optimum values. Final parameters should be selected using controlled experiments and a fixed validation/test split.

13.1 Retrieval / NLP Configuration

```

CHUNK_SIZE = 350 tokens
CHUNK_OVERLAP = 50
DENSE_TOP_K = 12
BM25_TOP_K = 12
GRAPH_TOP_K = 10
RRF_CANDIDATES = 20
RERANK_TOP_K = 20
FINAL_EVIDENCE_K = 6
MAX_RETRIEVAL_STEPS = 3

```

13.2 Confidence Configuration

```

ANSWER_SUPPORT_THRESHOLD = 0.80 # initial gate
WARNING_THRESHOLD = 0.65      # initial gate
MIN_EVIDENCE = 1
SYNTHESIS_MIN_EVIDENCE = 2

```

13.3 DQN Configuration

```

learning_rate = 1e-4
buffer_size = 50_000
learning_starts = 1_000
batch_size = 64
gamma = 0.95
train_freq = 4
gradient_steps = 1

```

```

target_update_interval = 2_000
exploration_initial_eps = 1.0
exploration_final_eps = 0.05
exploration_fraction = 0.20
policy_network = [128, 128]

```

13.4 Generator Configuration

```

context_tokens = 8192
temperature = 0.15
top_p = 0.85
top_k = 20
max_new_tokens = 1000

```

14. Speed and Resource Optimization

The system should be architected to use the local LLM once per question whenever possible. Retrieval and ranking should be bounded and parallelized.

Optimization	Implementation
Load models once	Keep embedding and LLM processes warm.
Cache PDF extraction	Never reparse the same document unless its hash changes.
Cache embeddings	Persist vectors keyed by chunk hash.
Parallel retrieval	Run FAISS, BM25 and graph retrieval concurrently after policy selection.
Rerank only 20	Do not rerank hundreds of candidates.
Keep final evidence small	5-7 chunks is the initial target.
Bound the RL horizon	3 retrieval decisions at first.
Use quantized GGUF	Reduces local memory pressure for the generator.
Avoid repeated LLM calls	Use deterministic gates and programmatic verification where possible.
Use SQLite	No separate database server required for the first version.

14.1 Target Performance Instrumentation

- PDF extraction time per page
- embedding time per 1,000 chunks
- FAISS search latency
- BM25 latency
- graph lookup latency
- reranker latency
- LLM generation latency
- end-to-end latency
- RAM usage
- VRAM usage
- tokens/second
- index size

15. Project Database and File Structure

Persistent Data Model

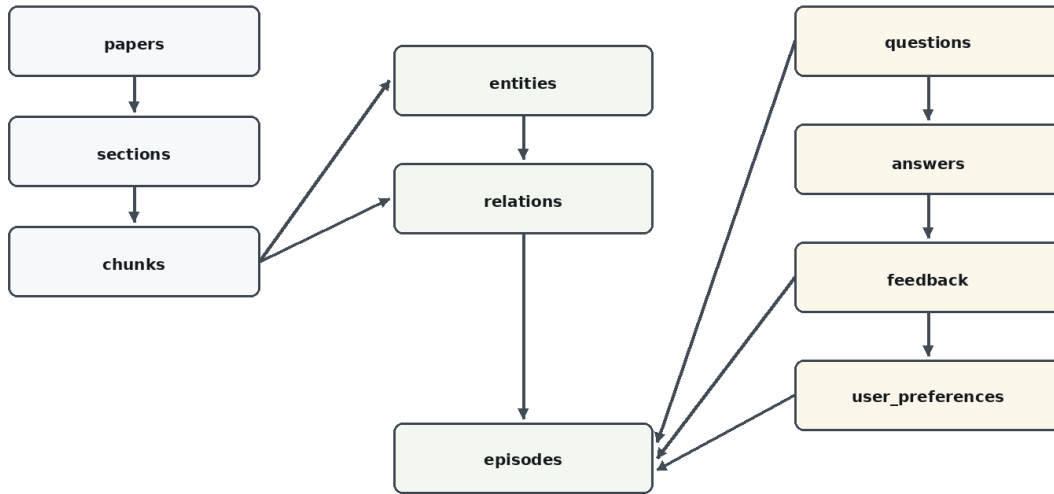


Figure 3. Persistent metadata and interaction data model. The vector index and graph are stored separately but referenced by stable IDs in SQLite.

15.1 SQLite Tables

Table	Key fields
papers	paper_id, file_hash, title, authors, year, path, page_count
sections	section_id, paper_id, title, level, page_start, page_end
chunks	chunk_id, paper_id, section_id, page, text, hash, vector_id
entities	entity_id, type, value, normalized_value
relations	subject_id, predicate, object_id, paper_id, page
questions	question_id, text, intent, timestamp
answers	answer_id, question_id, text, support_score, model_version
citations	answer_id, chunk_id, page, claim_id
feedback	question_id, answer_id, label, correction_text, timestamp
user_preferences	key, value, confidence, updated_at
rl_episodes	episode_id, question_id, state, action, reward, next_state

15.2 Recommended Repository Structure

```

ResearchVault/
├─ app/
├─ ingestion/
├─ nlp/
├─ retrieval/
├─ llm/
├─ verification/
├─ rl/

```

- └─ memory/
- └─ evaluation/
- └─ config/
- └─ models/
- └─ data/
- └─ tests/
- └─ requirements.txt
- └─ README.md

16. Evaluation Dataset and Experimental Methodology

16.1 Dataset Construction

1. Collect 20-50 research papers from a consistent technical domain for the first benchmark.
2. Create 300-500 questions with known evidence spans and answer references.
3. Include unanswerable questions whose answer is not present in the corpus.
4. Annotate the minimum evidence required for comparison and synthesis questions.
5. Split by paper when feasible: development and test papers should be disjoint.

16.2 Core Metrics

Category	Metrics
Retrieval	Recall@5, Recall@10, MRR, nDCG, Precision@K
QA	Exact Match where appropriate, token F1, human/LLM-verified correctness
Grounding	Citation accuracy, evidence support rate, unsupported-claim rate, hallucination rate
Abstention	Precision/recall of correctly refusing unanswerable questions
System	Latency, RAM, VRAM, tokens/sec, index size
RL	Mean reward, episode length, actions/question, convergence behavior, policy stability

16.3 Evaluation Protocol

1. Freeze the paper splits and benchmark questions before comparing systems.
2. Calibrate thresholds only on development data.
3. Run each baseline with identical retrieval corpus and evidence limits.
4. Run the RL policy with a fixed random seed set and multiple seeds where feasible.
5. Report mean and variance, not only one best run.
6. Perform ablations to identify which components create the gain.
7. Evaluate the held-out test set only after tuning is finished.

17. Baselines, Ablations and Research Questions

System	Description
A	Keyword/BM25 only

B	Dense vector RAG
C	Dense + BM25
D	Dense + BM25 + reranker
E	Dense + BM25 + graph + reranker
F	RL-adaptive retrieval + all components

17.1 Key Research Questions

- Does hybrid retrieval improve evidence recall over dense or lexical retrieval alone?
- Does a reranker improve answer faithfulness enough to justify its latency cost?
- Does graph retrieval materially help comparison and synthesis questions?
- Does an RL policy improve answer correctness or reduce retrieval steps relative to a fixed retrieval pipeline?
- Does user feedback improve personalized retrieval decisions over time?
- Can an abstention gate reduce unsupported claims without making the system unusably conservative?

17.2 Recommended Ablations

Ablation	Remove / change	What it tests
No BM25	Dense only	Value of lexical exact-match search.
No graph	Vector + BM25 only	Value of structured relationships.
No reranker	Use fusion ranking only	Value of second-stage relevance ranking.
No RL	Fixed hybrid policy	Incremental value of adaptive policy.
No feedback	Ignore user corrections	Effect of personalization/feedback loop.
Larger LLM	2B vs 4B	Generator size vs retrieval quality.

18. UI / UX Design and Key Features

RESEARCHVAULT

[+ Upload PDF] [Corpus: 127 papers] [Concepts: 4,821]

Ask your research knowledge base...

Answer

...

Evidence

[1] Paper A - Page 7 - Results

[2] Paper C - Page 12 - Method

Confidence: High

[Why this answer?] [Correct] [Incorrect] [Correct the answer]

18.1 Key Features

- Drag-and-drop PDF ingestion with processing progress.
- Corpus browser with papers, sections, entities, methods, datasets and metrics.
- Question answering with source/page citations.
- "Why did you say this?" evidence inspector.
- User feedback buttons and correction capture.
- Confidence indicator and explicit abstention.
- Cross-paper comparison mode.
- Research-gap / limitations view.
- Offline status indicator and model/index health status.

19. Research-Gap Detection and Advanced Extensions

Once the basic knowledge graph and cross-paper retrieval are working, ResearchVault can become more than a QA system. It can analyze the stored literature to identify repeated limitations, understudied datasets, contradictory findings, rarely compared methods, and frequently suggested future work.

19.1 Research-Gap Pipeline

Papers -> Extract limitations/future work -> Normalize concepts -> Cluster repeated gaps -> Compare with methods/datasets/results -> Surface recurring gaps -> Cite supporting papers

19.2 Advanced RL Directions

- Contextual bandits for very fast retrieval strategy selection.
- Double/Dueling/Prioritized variants if vanilla DQN plateaus and the environment design supports them.
- Offline RL when large logged interaction datasets are available.
- Meta-RL for rapid adaptation to new users or research domains.
- Safe RL constraints to prevent unnecessary retrieval loops or unsupported answering.
- Multi-agent retrieval where separate agents specialize in lexical, semantic, graph, and verification tasks.

19.3 Potential Dissertation Extension

Suggested research contribution

Study whether RL-based retrieval strategy selection improves citation-supported QA accuracy and retrieval efficiency in a fully offline research knowledge system, while preserving low memory and low latency on commodity hardware.

20. Step-by-Step Build Roadmap

Phase	Deliverable
Phase 1 - Offline NLP Reader	PDF -> structured Markdown/JSON -> sections -> entities -> cached files.

Phase 2 - Research Memory	Semantic chunks -> embeddings -> FAISS -> BM25 -> SQLite metadata.
Phase 3 - Grounded QA	Query -> hybrid retrieval -> reranker -> local LLM -> citations.
Phase 4 - Reliability	Confidence calibration -> citation checker -> numeric checker -> abstention.
Phase 5 - Knowledge Graph	Build entities/relations and graph-based retrieval.
Phase 6 - Personalization	Store preferences, corrections and interaction history.
Phase 7 - RL Environment	Define state, actions, transitions, rewards, max 3 steps.
Phase 8 - DQN Training	Train on development queries; validate policy improvements.
Phase 9 - Research Intelligence	Cross-paper comparisons, trend analysis, research-gap detection.
Phase 10 - Dissertation Benchmark	Freeze benchmark, run baselines/ablations, document results.

20.1 Recommended Development Order for a Single Developer

1. Get one clean paper through the parser and preserve page/section metadata.
2. Build deterministic chunking and embeddings.
3. Implement FAISS + BM25 retrieval and inspect results manually.
4. Integrate the local LLM only after retrieval is trustworthy.
5. Add citations and verification before adding RL.
6. Create and freeze an initial validation set.
7. Implement the RL environment on top of the stable retrieval engine.
8. Train DQN and compare against the fixed hybrid baseline.
9. Add personalization and research-gap analysis last.

21. Risk, Failure Modes and Recovery Strategy

Failure mode	Likely cause	Mitigation
Wrong answer with good-looking citation	Retriever found related but non-supporting text.	Reranker + claim/evidence check + citation verification.
Hallucinated number	LLM generated plausible statistic.	Require exact numeric evidence match.
Correct answer not retrieved	Poor chunking or query mismatch.	Hybrid retrieval + query expansion + better chunk boundaries.
Too many false abstentions	Threshold too high or calibrator poor.	Tune on validation set and inspect precision/recall trade-off.
RL learns shortcuts	Reward leaks from weak proxy metrics.	Use claim-level reward and held-out evaluation.
Slow response	Too many retrieval steps or oversized context.	Parallel retrieval, capped K, one LLM generation.
Memory growth	Repeated embeddings/logging.	Hash-based deduplication and retention policies.
Scanned papers fail	No text layer.	Selective OCR with cached output.

21.1 Security / Privacy Design

- Keep the research corpus in local directories controlled by the application.

- Do not send paper text to external APIs in offline mode.
- Log only what is necessary for evaluation and debugging.
- Use file hashes to identify document versions and avoid duplicate indexing.
- Provide a clear local-only indicator and disable network-dependent features by default.

22. Deliverables and Dissertation-Ready Outputs

Deliverable	Contents
Working application	Local desktop/web UI for ingestion, retrieval and QA.
Model package	Pinned embedding, reranker and local generator versions.
Knowledge store	SQLite DB, FAISS index, graph files and cached extraction artifacts.
Evaluation suite	Benchmark questions, evidence labels, scripts and metrics.
RL environment	Gymnasium environment and reproducible training configuration.
Experiment report	Baselines, ablations, accuracy, grounding, latency and resource usage.
Research paper / dissertation	Problem, methodology, experiments, limitations and future work.
Demo dataset	A small set of papers and questions for repeatable demonstration.

22.1 Suggested Final Project Title

ResearchVault: An Offline NLP and Reinforcement Learning-Based Persistent Research Knowledge Agent for Adaptive, Citation-Grounded Question Answering

22.2 Suggested Abstract Direction

The project proposes an offline research intelligence agent that transforms uploaded scientific literature into persistent semantic and structured knowledge. Natural language processing is used for document understanding, section-aware chunking, entity and relation extraction, semantic representation, and grounded question answering. A local vector index, lexical retrieval system, and knowledge graph provide complementary evidence sources. Reinforcement learning is formulated as a sequential retrieval-strategy selection problem, where the agent learns to choose evidence-gathering actions based on query intent, retrieval state, user context, and feedback. The system emphasizes citation traceability, confidence calibration, abstention, low hallucination rates, and low-resource local inference. Experimental evaluation compares fixed retrieval baselines with the RL-adaptive system across retrieval quality, answer correctness, citation accuracy, hallucination rate, latency, and memory usage.

23. References and Official Documentation

PyMuPDF4LLM documentation

<https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/>

PDF extraction, layout-aware parsing, JSON/Markdown output and selective OCR.

PyMuPDF4LLM API

<https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/api.html>

API details including page chunks and layout box metadata.

BAAI/bge-small-en-v1.5 model card

<https://huggingface.co/BAAI/bge-small-en-v1.5>

Local embedding model and usage guidance.

FAISS documentation - IndexHNSW

https://faiss.ai/cpp_api/file/IndexHNSW_8h.html

HNSW index structure and search API.

Stable-Baselines3 DQN documentation

<https://stable-baselines3.readthedocs.io/en/master/guide/algos.html>

Supported algorithms and discrete-action guidance.

Stable-Baselines3 DQN API

<https://stable-baselines3.readthedocs.io/en/master/modules/dqn.html>

DQN configuration and parameters.

llama.cpp README

<https://github.com/ggml-org/llama.cpp/blob/master/README.md>

Local inference, quantization, CUDA and CPU+GPU hybrid execution.

llama.cpp server documentation

<https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>

Local server, OpenAI-compatible API, JSON constraints and inference options.

Qwen model family documentation

<https://huggingface.co/Qwen>

Model-family releases; pin the exact model and revision used in experiments.

Source verification note: model and library capabilities can change. Pin exact package/model versions in the project environment and record the date and commit/revision used for every benchmark run.